

Autómatas, Teoría de Lenguajes y Compiladores

Trabajo Práctico Especial

Instituto Tecnológico de Buenos Aires

Grupo G94

Integrantes:

Sol Medina (63567)

Santiago Sanchez Marostica (64056)

Martín Gallardo Bertuzzi (63510)

Índice

1. Introducción	2
2. Modelo Computacional	2
2.1. Dominio	2
2.2. Lenguaje	3
3. Implementación	4
3.1. Frontend	5
3.2. Backend	6
3.2.1. Validación Semántica	6
3.2.2. Generación de Código	7
3.3. Dificultades Encontradas	7
4. Futuras Extensiones	8
5. Conclusiones	8
6. Bibliografía	9

1. Introducción

Este trabajo consistió en armar un compilador que le permita a un usuario escribir una presentación usando un lenguaje de texto plano bastante simple, y que luego eso se transforme automáticamente en un formato de presentación válido, en este caso en lenguaje LaTeX Beamer. La idea del lenguaje es que el usuario pueda enfocarse en el contenido de su presentación y que directamente se genere una presentación ya hecha.

El compilador se hizo en lenguaje C, usando Flex para el análisis léxico y Bison para el análisis sintáctico. Con eso se pudo reconocer las partes del lenguaje de entrada y validarlas con la gramática que se definió, para después armar una estructura interna que permita generar la presentación final.

En general, se dividió el proceso en tres partes: primero definir el dominio y qué entrada necesitaba el lenguaje, después hacer todo el frontend (léxico y sintaxis), y por último implementar el backend, donde se hicieron las validaciones semánticas y la generación del código final.

El proyecto se construyó sobre la base del template [Flex-Bison-Compiler](#) provisto por la cátedra, lo que permitió aprovechar una arquitectura bien definida con 12 módulos principales distribuidos entre frontend, backend y sistema de soporte. El proceso de compilación utiliza CMake como sistema de build, con opciones de optimización (`-O3`) y sanitización de memoria (`-fsanitize=address`) para garantizar robustez y detectar errores en tiempo de desarrollo.

2. Modelo Computacional

Como parte del diseño del compilador, se definieron: el **dominio** del lenguaje y el **lenguaje de entrada** que el compilador va a aceptar.

2.1. Dominio

El dominio del proyecto estuvo enfocado en armar presentaciones partiendo de texto plano. La idea fue diseñar un texto simple que permitiera definir:

- Slides con títulos
- Listas de ítems numeradas o con bullets
- Imágenes
- Subtítulos
- Bloques de código
- Notas del orador
- Bloques especiales (alert, example, normal)
- Links entre diapositivas

La idea principal fue dar una opción más simple que escribir directamente en LaTeX Beamer, de manera que alguien sin experiencia en este lenguaje pueda, de igual manera, armar una presentación completa de manera fácil y sencilla.

El dominio también incluyó las restricciones propias del formato como una imagen sin leyenda, títulos sin espacios o tipos de bloque desconocidos, y eso fue clave para definir qué casos había que rechazar y qué validaciones hacer después.

Para representar internamente estos elementos, se diseñó un Árbol de Sintaxis Abstracta (AST) con estructuras de datos específicas definidas en `AbstractSyntaxTree.h`:

- **Text**: Contenido textual con tipo de lista (bullets, numerada o sin formato)
- **Image**: Path, caption obligatorio y legend opcional
- **CodeBlock**: Contenido de código preservando formato original
- **Note**, **Block**, **Link**: Elementos especiales con sus campos específicos
- **SlideItem**: Unión discriminada con puntero `next` para listas enlazadas
- **Slide**: Diapositiva con título, subtítulo, ID e items opcionales
- **Program**: Nodo raíz con lista enlazada de slides

Cada estructura tiene funciones destructoras que liberan recursivamente la memoria asignada, verificadas con el sanitizador de GCC (`-fsanitize=address`).

2.2. Lenguaje

El lenguaje de entrada se basó en una sintaxis simple de texto plano, con marcadores simples:

- `#` para títulos
- `##` para subtítulos
- `-` para bullets
- `1. 2. 3.` para listas numeradas
- `@img "pathcaption"` para imágenes (con `"legend.opcional"` como tercer parámetro)
- `@code ... @end` para bloques de código
- `@note "texto"` para notas del orador
- `@block tipo=... "Título"... @end` para bloques especiales (el título es opcional)
- `---` para separar slides
- `@link "DestinoTexto"` para links entre diapositivas (el texto es opcional)
- `@id identificador` para asignar un ID único a un slide

También se definieron comentarios con `/* ... */`.

A nivel sintáctico, el lenguaje permite:

- Un slide por título
- Inclusión libre de bloques e imágenes dentro de un slide
- Secciones opcionales mediante subtítulos
- Estructuras bien balanceadas (`@code` y `@block` deben cerrarse)
- Links para moverse fácilmente entre diapositivas

Los casos inválidos también fueron parte importante del diseño, como componentes desconocidos, bloque de código sin cerrar, comentarios abiertos o títulos sin espacios.

La gramática implementada en Bison define 12 no-terminales principales que estructuran el lenguaje, desde el nodo raíz *program* hasta los elementos individuales como *text*, *image*, *codeBlock*, etc.

El scanner de Flex reconoce más de 20 tokens diferentes que incluyen marcadores de estructura (`#`, `-`, `---`), directivas (`@img`, `@code`, `@note`, `@block`, `@link`, `@id`), tipos de bloque (NORMAL, ALERT, EXAMPLE), cadenas de texto y contenido libre.

Para manejar correctamente el contenido dentro de bloques sin que los tokens se interpreten erróneamente, el scanner utiliza 4 contextos exclusivos de Flex:

- **MULTILINE_COMMENT**: Ignora todo entre `/*` y `*/`
- **CODE_BLOCK**: Captura literalmente todo entre `@code` y `@end`, incluyendo caracteres especiales
- **TEXT_LINE**: Captura contenido de listas hasta fin de línea
- **BLOCK_CONTENT**: Captura contenido dentro de bloques `@block`

Este manejo permite que dentro de un bloque de código se pueda escribir `@img "test"` sin que se interprete como directiva.

3. Implementación

Para la implementación se usó de base el proyecto provisto por la cátedra [Flex-Bison-Compiler](#). Esto facilitó el uso de todas las funcionalidades de Flex (análisis léxico) y las funcionalidades de Bison (análisis sintáctico).

3.1. Frontend

En el frontend se definió la gramática que representaba el lenguaje diseñado. Se identificaron como terminales los distintos símbolos: `#`, `-`, `@img`, `@code`, `@end`, `@note`, `@block`, `@link`, cadenas, comentarios, etc.

Como no terminales se definieron:

- La estructura general de la presentación
- Los slides
- Los ítems
- Los bloques especiales
- Las imágenes
- Los componentes de cada slide

Durante el análisis sintáctico se validaron numerosas restricciones del lenguaje:

- Un bloque que abre debe cerrarse con `@end`
- El componente `@img` debe tener path y caption
- Solo se aceptan los tipos de bloque: `normal`, `alert` o `example`

El análisis léxico en `FlexPatterns.1` utiliza expresiones regulares específicas. Las transiciones entre contextos se manejan con `BEGIN()`: al encontrar `@code` se ejecuta `BEGIN(CODE_BLOCK)`, y con `@end` se vuelve al contexto inicial.

Dentro del contexto `CODE_BLOCK`, Flex usa reglas ordenadas para capturar correctamente el contenido, incluyendo secuencias como `@import` que comienzan con `@` pero no son `@end`.

Cada acción léxica copia el lexema con `strdup()`, actualiza la posición en el archivo y retorna el token correspondiente.

Las reglas gramaticales en `BisonGrammar.y` permiten múltiples variaciones de slides: con solo título, con título y subtítulo, con identificador personalizado, o sin título. Cada regla ejecuta una acción semántica específica implementada en `BisonActions.c` que aloca memoria con `calloc()`, copia las cadenas con `strdup()` y retorna el nodo creado.

El tipo `SemanticValue` permite que cada símbolo tenga su tipo específico (`STRING` como `char *`, slide como `Slide *`). La gramática incluye destructores automáticos (`%destructor`) que liberan memoria si ocurre un error, evitando memory leaks.

Durante la revisión final se realizaron algunas simplificaciones en el frontend para mejorar la claridad de la gramática. En particular, se factorizaron reglas que presentaban prefijos repetidos, como las variantes de `slide`, integrando estos patrones dentro de no terminales auxiliares. Esto permitió reducir ambigüedad estructural y

mejorar la legibilidad en Bison sin modificar el lenguaje aceptado.

Originalmente, el parser recibía cadenas entre comillas y debían limpiarse mediante la función `_removeQuotes`. Siguiendo la recomendación docente, esta responsabilidad fue trasladada al lexer: ahora Flex entrega directamente el contenido sin comillas. Esto elimina procesamiento redundante en Bison, reduce complejidad y simplifica las acciones semánticas asociadas.

Otra mejora consistió en mover la lógica de interpretación del tipo de bloque (`normal`, `alert`, `example`) desde Bison al lexer. Flex ahora reconoce estas palabras clave y genera tokens específicos, eliminando la necesidad de la función `_stringToBlockType`. Esto evita una forma implícita de backtracking y deja la semántica completamente determinada antes de entrar al parser.

3.2. Backend

El backend se encarga de las validaciones que no se pueden hacer solo con la gramática, especialmente aquellas que requieren ver la estructura completa de la presentación o las relaciones entre diferentes elementos.

Las validaciones principales incluyen:

- Las imágenes deben incluir ambos argumentos (ruta y descripción)
- No se pueden utilizar componentes desconocidos (por ejemplo `@foo`)
- Los bloques deben tener un tipo válido (`normal`, `alert` o `example`)
- Los bloques `@code` y `@block` deben cerrarse correctamente con `@end`
- Los links internos deben apuntar a slides existentes
- Las diapositivas pueden no tener título (para continuaciones)

Una vez que todo está validado, el backend genera el código LaTeX Beamer que produce la presentación final.

3.2.1. Validación Semántica

El analizador semántico (`SemanticAnalyzer.c`) trabaja en dos pasos:

Primer paso: Recorre todas las diapositivas y va guardando una lista de IDs y títulos. Mientras tanto, verifica que no haya IDs repetidos, que las imágenes tengan descripción, y que los bloques usen tipos válidos.

Segundo paso: Verifica que cuando un link apunta a otra diapositiva, esa diapositiva realmente exista. Este proceso en dos pasos es necesario porque un link puede referenciar una diapositiva que aparece más adelante en el documento.

3.2.2. Generación de Código

El generador (`Generator.c`) realiza varias tareas importantes:

- Escapa caracteres especiales de LaTeX (como `_`, `%` y `$`) para que se muestren correctamente
- Convierte los títulos en etiquetas válidas para los links internos (todo en minúsculas, sin espacios ni caracteres especiales)
- Formatea el código según preferencias configurables
- Crea un mapeo entre los IDs del usuario y las etiquetas internas de Beamer
- Identifica qué diapositivas necesitan etiquetas para poder ser referenciadas

El proceso completo genera un documento LaTeX Beamer válido: imprime el encabezado necesario, procesa cada diapositiva generando los comandos apropiados según el tipo de contenido (texto, imágenes, código, notas, bloques, links), y cierra el documento. El resultado se envía directamente a la salida estándar.

3.3. Dificultades Encontradas

Las principales dificultades incluyeron:

- **Bloques anidados:** Los bloques `@code` y `@block` requieren reglas especiales para balanceo y captura de contenido
- **Contextos de Flex:** Capturar correctamente secuencias como `"@e"`, `"@en"` dentro de código sin confundirlas con `"@end"`. Se resolvió con reglas ordenadas por especificidad
- **Resolución de links:** Implementar mapeo bidireccional donde IDs y títulos apuntan al mismo label de Beamer, permitiendo referencias flexibles
- **Preservación de formato:** Mantener exactamente la indentación y espaciado del código fuente, configurable mediante variables de entorno (original vs auto-format)
- **Comentarios multilínea:** Manejar correctamente `/*...*/` incluso con contenido especial o múltiples líneas

Durante las pruebas de memoria se identificó una pérdida muy pequeña (35 bytes) que ocurre únicamente en caso de entrada inválida. Tras el análisis, se comprobó que este leak no proviene del AST ni de las acciones semánticas, sino del manejo de errores interno de Bison: cuando una producción falla sin llegar a reducirse, el parser no ejecuta los destructores asociados a los valores que ya estaban en la pila.

Este comportamiento está documentado en el manual de Bison y es una limitación conocida del modelo LALR sin reglas de recuperación de errores. Solucionarlo requeriría reescribir la gramática para incorporar mecanismos de error recovery, lo

cual es complejo y no aporta beneficios prácticos al compilador.

En las ejecuciones válidas (17/17 casos) y en la mayoría de las entradas inválidas (6/7 casos), el compilador libera completamente la memoria. El leak restante ocurre solo en casos de error sintáctico y es considerado aceptable dentro del alcance del proyecto.

Por otro lado, en las correcciones del front se señaló que sería deseable flexibilizar el manejo de espacios en ciertos patrones del lexer, sin embargo, debido al tiempo reducido entre la devolución y la fecha de entrega no llegamos a implementarla sin comprometer al resto del compilador.

4. Futuras Extensiones

Si bien se cumple con los objetivos iniciales, existen varias direcciones en las que podría expandirse:

- Agregar soporte para animaciones o transiciones
- Permitir tablas o diagramas
- Soportar incrustación de video
- Generar directamente un PDF final sin que el usuario compile nada

Estas mejoras harían que el lenguaje sea aún más flexible y se acerque más a un sistema completo de escritura de presentaciones.

Adicionalmente, sería valioso implementar:

- **Múltiples backends:** Además de LaTeX Beamer, generar HTML (reveal.js), Markdown o PowerPoint, reutilizando el mismo frontend y AST
- **Mensajes de error mejorados:** Reportar números de línea específicos usando las capacidades de tracking de Bison (`@1`, `@2`, etc.)
- **Temas personalizables:** Permitir seleccionar temas de Beamer mediante directivas en el archivo fuente
- **Sintaxis para ecuaciones:** Agregar marcadores como `@eq` para fórmulas matemáticas sin necesidad de escribir LaTeX directamente

5. Conclusiones

Se logró completar el desarrollo de un compilador para armar presentaciones, trabajando en el diseño del lenguaje, frontend y backend. El trabajo permitió aplicar en la práctica análisis léxico, sintáctico y semántico usando Flex y Bison.

El sistema de testing valida el compilador mediante 24 casos de prueba: 17 tests de aceptación (slides básicos, imágenes, código, notas, bloques especiales, subtítulos, listas, links internos) y 7 tests de rechazo (imágenes sin caption, bloques sin cerrar, componentes desconocidos, títulos mal formados, comentarios abiertos). El script

`test.sh` automatiza la ejecución completa usando el código de salida del compilador.

Arquitectura:

El código está organizado en 12 módulos principales: EntryPoint, Frontend, FlexActions, BisonActions, AbstractSyntaxTree, SemanticAnalyzer, Generator, Logger, String, Environment. Cada módulo expone una interfaz clara ocultando detalles de implementación.

El sistema de logging soporta 6 niveles (ALL, DEBUGGING, INFORMATION, WARNING, ERROR, CRITICAL) configurables mediante variable de entorno. La gestión de memoria utiliza destructores automáticos de Bison y el sanitizador de GCC (`-fsanitize=address`) para detectar leaks, use-after-free y buffer overflows.

El proyecto demuestra cómo un lenguaje de dominio específico bien diseñado simplifica tareas complejas, permitiendo crear presentaciones profesionales sin experiencia en LaTeX.

6. Bibliografía

1. (2025-08-21, v3.0.8) Proyecto Especial. <https://docs.google.com/document/d/1XZy63UwUVHyMCR6p051aer-i9SpbLmo0SGC-gh6Cb-I/edit?tab=t.0>
2. (2024-05-16, v0.1.0) Generación de Código. https://docs.google.com/document/d/1gTDatettvr3GLewUy4_iwacQvTCZAaks02xu6l6eFcc/edit?tab=t.0#heading=h.detc7r7352wx
3. (2024-09-25, v0.2.0) Análisis Semántico. https://docs.google.com/document/d/1ZqUt72P0tTA-IXPHUVgFqz0hbb7-QQnc5i_QOB1rMPY/edit?tab=t.0#heading=h.detc7r7352wx
4. (2024-05-08, v0.1.0) Análisis Sintáctico. https://docs.google.com/document/d/1HNNtofu_TCHadDxham2K-_HwKfaCG55izvX8a-QtqZU/edit?tab=t.0#heading=h.detc7r7352wx
5. (2025-09-03, v1.0.0) Análisis Léxico. <https://docs.google.com/document/d/1o0rYT3vbLtdxuMNnIirf7je2bp2B6FuPGiwCJkVrx0g/edit?tab=t.0#heading=h.detc7r7352wx>
6. (2025-03-13, v0.1.0) Modelo Computacional. https://docs.google.com/document/d/1V6Ds0d9hhpiCoMlqdqVNyaVvW0mIZ8ylNW4TD_rruDY/edit?tab=t.0#heading=h.detc7r7352wx
7. GNU Bison Manual, Section 3.7.7 - https://www.gnu.org/software/bison/manual/html_node/Decl.html